Ders 10 Lab Çalışma Özeti

Ders 10 Lab Çalışma Özeti

Genel Konular

- **Record (Kayıt) Tipi Tanımlama**
  - CREATE TYPE ifadesi ile yeni veri tipleri oluşturma
  - Birden fazla veriyi tek bir return ifadesinde döndürme ihtiyacı
  - Fonksiyon dışında tanımlanması ve birden fazla fonksiyonda kullanılabilmesi
  - Tip tanımı örnekleri: çalışan (isim, soyisim, maaş), ürünler (miktar1, miktar2)
- **Cursor (İmleç) Kullanımı**
  - DECLARE kısmında cursor tanımlama (CURSOR FOR ile sorgu yazma)
  - Sorgu sonucu oluşan tablonun satır satır okunması
  - FOR satır IN cursor_name LOOP yapısı ile döngüye sokma
  - Her bir satır üzerinde koşullu işlem yapabilme (IF ile kontrol)
  - SUM fonksiyonu yerine cursor ile toplam hesaplama örneği
- **Trigger (Tetikleyici) Tanımları**
  - BEFORE ve AFTER trigger farkları ve kullanım alanları
  - INSERT, UPDATE, DELETE işlemleri için tetikleme
  - FOR EACH ROW (her satır için) ve FOR EACH STATEMENT (bir kere çalışma) farkı
  - RETURN NEW, RETURN OLD, RETURN NULL dönüş değerleri
  - Trigger fonksiyonu oluşturma (RETURNS TRIGGER zorunluluğu)
- **PL/pgSQL Array (Dizi) Kullanımı**
  - Dizi tanımı ve indeksleme (1'den başlar, sıfırdan başlamaz)
  - Cursor ile doldurulan diziye kayıt ekleme
  - Dizi indeksini artırarak sıradaki elemana geçme

Hocanın Özellikle Vurguladığı Kısımlar

- **CREATE OR REPLACE FUNCTION kullanımı**
  - Sadece CREATE FUNCTION dendiğinde fonksiyon zaten varsa hata verir
  - REPLACE eklenmesi, fonksiyonda değişiklik yapıldığında güncellenmesini sağlar
- **Parametre tanımlama sırası**
  - PL/pgSQL'de önce değişken adı, sonra tipi yazılır (programlama dillerinin tersine)
  - RETURN tipi parantez dışında belirtilir
- **RAISE ifadesi ile mesaj yazdırma**
  - RAISE NOTICE: Bilgilendirme mesajı
  - RAISE EXCEPTION: Hata mesajı (error olarak döndürür)
  - RAISE INFO, WARNING: Diğer mesaj türleri
  - Hiçbir şey yazılmazsa default olarak exception döner
- **Record tipinin fonksiyon dışında tanımlanması**
  - Create type fonksiyon bloğunun dışında yapılır
  - Böylece farklı fonksiyonlarda aynı tip kullanılabilir
- **Cursor ile satır satır okuma mantığı**
  - Bir sorgu sonucu tablo dönüyorsa ve bu tablonun satırları üzerinde işlem yapılacaksa cursor kullanılır
  - For döngüsü ile her satır tek tek incelenir
  - Koşullara göre filtreleme, hesaplama veya güncelleme yapılabilir
- **OLD ve NEW keyword'leri**
  - INSERT işlemlerinde sadece NEW kullanılır (yeni değer)

  - DELETE işlemlerinde sadece OLD kullanılır (eski/silinen değer)
  - UPDATE işlemlerinde hem OLD hem NEW kullanılır (eski ve yeni değer)
- **Trigger silme işlemi**
  - Hem trigger'ın kendisi hem de trigger fonksiyonu ayrı ayrı silinmelidir
  - DROP TRIGGER ve DROP FUNCTION birlikte yapılmalıdır
- **Constraint (Kısıt) yönetimi**
  - Trigger ile veri tabanı değişikliği yapılacaksa, önceden tanımlı kısıtlar kaldırılmalıdır
  - ALTER TABLE ile DROP CONSTRAINT yapılarak kısıtlar silinir
- **TG_OP (Trigger Operation) kullanımı**
  - Birden fazla DML komutu ile tetiklenen trigger'larda hangi işlemin gerçekleştiği kontrol edilir
  - IF TG_OP = 'DELETE' THEN ... ELSIF TG_OP = 'UPDATE' THEN ... ELSE (INSERT) yapısı
- **For Each Row ve For Each Statement farkı**
  - FOR EACH ROW: Etkilenen her satır için ayrı ayrı çalışır
  - FOR EACH STATEMENT: Sorgu bir kere çalışır, satır sayısı önemli değildir

Kısa Tekkrar Notları

- Fonksiyon tanımı: CREATE OR REPLACE FUNCTION fonksiyon_adı(parametre tipi) RETURNS dönüş_tipi AS
$$BEGIN...END$$
LANGUAGE plpgsql;
- Parametreler: parametre_adı tipi şeklinde (değişken adı önce, tipi sonra)
- DECLARE kısmında fonksiyon içi değişkenler tanımlanır
- BEGIN ve END arasında ana mantık yazılır
- RETURN ile değer döndürülür veya OUT parametresi kullanılır
- CREATE TYPE ile yeni tip tanımlanır (fonksiyon dışında)
- CURSOR FOR ile sorgu sonucu tablo satır satır okunur
- FOR satır IN cursor LOOP ile döngüye girilir
- Trigger fonksiyonu RETURNS TRIGGER döndürmelidir
- Trigger tanımlaması: CREATE TRIGGERtrigger_adı BEFORE/AFTER INSERT/UPDATE/DELETE ON tablo FOR EACH ROW EXECUTE PROCEDURE fonksiyon();
- Trigger silme: DROP TRIGGERtrigger_adı ON tablo; ve DROP FUNCTION fonksiyon_adı;
- RAISE NOTICE/EXCEPTION/INFO ile mesaj yazdırılır
- PL/pgSQL array indeksleri 1'den başlar

Detaylı Açıklamalar

Record (Kayıt) Tipi Tanımlama

Record tipi, birden fazla veriyi tek bir yapıda toplamak için kullanılır. Normalde bir fonksiyon tek bir değer döndürürken, record tipi ile ad, soyad ve maaş gibi birden fazla değer aynı anda döndürülebilir. CREATE TYPE komutuyla yeni bir tip tanımlanır ve bu tip fonksiyon dışında, dolayısıyla farklı fonksiyonlarda da kullanılabilir şekilde oluşturulur. Örneğin "çalışan" adlı bir tip oluşturup içine isim (varchar), soyisim (varchar) ve maaş (integer) alanları tanımlanabilir. Fonksiyon bu tipte bir değişken return ettiğinde, üç değer de birlikte döner. Fonksiyon çağrılırken SELECT fonksiyon_adı(ssn_değeri) şeklinde, dönen sonuçta bu üç alan da görüntülenir.

Cursor (İmleç) Kullanımı

Cursor, bir sorgu sonucu oluşan tablonun satırlarını tek tek okumak ve üzerinde işlem yapmak için kullanılır. DECLARE bölümünde cursor adı ve FOR ile sorgu tanımlanır. Sonra BEGIN bloğunda FOR satır IN cursor_adı LOOP yapısıyla döngüye girilir. Her döngüde cursor'ın gösterdiği satır okunur ve satır.column_name şeklinde erişilir. Örneğin bir departmanın çalışanlarının toplam maaşını hesaplarken, SUM fonksiyonu kullanmak yerine cursor ile her bir çalışanın maaşı tek tek toplanabilir. Bu yöntem, daha karmaşık koşullu işlemler için de kullanılabilir (örneğin maaşı belirli bir değere tam bölünenlerin filtrelenmesi). Dizi (array) ile birlikte kullanılarak, koşulları sağlayan kayıtlar bir dizide toplanabilir. PL/pgSQL'de dizilerin indeksleri 1'den başlar.

Trigger (Tetikleyici) Tanımları

Trigger'lar, veri tabanında INSERT, UPDATE veya DELETE işlemi yapıldığında otomatik olarak çalışan özel fonksiyonlardır. İki aşamada oluşturulur: önce trigger fonksiyonu (RETURNS TRIGGER), sonra trigger'ın kendisi. Trigger fonksiyonu BEGIN ve END arasında tetiklendiğinde yapılacak işlemleri barındırır. RETURN NEW ile yeni değer, RETURN OLD ile eski değer, RETURN NULL ile işlem iptali sağlanır. BEFORE trigger'ı, işlem veri tabanına yazılmadan önce; AFTER trigger'ı ise yazıldıktan sonra çalışır. FOR EACH ROW, etkilenen her satır için; FOR EACH STATEMENT ise sadece bir kere çalışır. Birden fazla DML komutu ile tetiklenen trigger'larda TG_OP değişkeni ile hangi işlemin gerçekleştiği kontrol edilir (IF TG_OP = 'DELETE' THEN ... ELSIF TG_OP = 'UPDATE' THEN ... ELSE INSERT END IF). Trigger örnekleri arasında: tatil günlerinde ve mesai saatleri dışında insert engelleme, departman numarası güncellendiğinde employee tablosunda da güncelleme yapma, maaş azaltılmasına ve %10'dan fazla artışa izin vermeme, departman tablosuna toplam maaş sütunu ekleyip employee değişikliklerinde otomatik güncelleme sayılabilir.

Trigger ile Veri Bütünlüğü Sağlama

Departman ve employee tabloları arasındaki ilişkide, departman numarası güncellendiğinde employee tablosundaki ilgili kayıtların da güncellenmesi gerekir. Ancak veri tabanında zaten foreign key kısıtları tanımlı olabilir. Trigger kullanmadan önce bu kısıtların ALTER TABLE ile DROP CONSTRAINT ile kaldırılması gerekir, aksi halde güncelleme işlemi yapılamaz. Kısıtlar kaldırıldıktan sonra AFTER UPDATE trigger'ı ile department tablosundaki değişiklik, employee tablosuna otomatik olarak yansıtılır. Bu sayede veri bütünlüğü korunur.

Trigger Fonksiyonunda TG_OP Kullanımı

Birden fazla DML komutu (INSERT, UPDATE, DELETE) ile tetiklenen bir trigger fonksiyonunda, hangi işlemin gerçekleştiği TG_OP kontrolü ile anlaşılır. DELETE işleminde old.saları ile silinen maaş toplamdan çıkarılır; INSERT işleminde new.saları toplama eklenir; UPDATE işleminde ise hem old.saları çıkarılıp hem new.saları eklenerek güncelleme yapılır. Bu sayede tek bir trigger fonksiyonu ile birden fazla işlem türü yönetilebilir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.